Campus Ride-Pooling and Split-Fare System

Deployment Documentation

Course: CS455
Deployment platform: Render.com Free tier
Application: Campus Ride-Pooling and Split-Fare System
Repository: `CS455-Carpooling/Campus_Ridepooling_Automation`

1. Deployment Overview

The application is deployed as a continuously running Docker-based web service on Render.com. A Render Managed PostgreSQL Free instance provides the production database. The deployment preserves the same application container, database naming, schema migration process, and runtime model used during local development.

The production deployment consists of:

- A Render Web Service running the application container.
- A Render Managed PostgreSQL Free database.
- A private-network connection from the web service to PostgreSQL using Render's Internal Database URL.
- Automatic deployments triggered by pushes to the connected GitHub branch.

The application is a Next.js web application with a custom Node.js server and Socket.IO support. Render runs the container as a web service, allowing the real-time ride matching, ride chat, and notification channels to remain available while the service is awake.

2. Architecture and Environment Parity

2.1 Local architecture

The local deployment is defined in `docker-compose.yml`. It contains:

- A PostgreSQL 16 service named `db`.
- A web service built from `apps/web/Dockerfile`.
- A persistent Docker volume for PostgreSQL data.
- A health check that waits for PostgreSQL before starting the web service.
- A web health check against `/api/health`.

The local PostgreSQL configuration uses:

Purpose	Database	User	Port
Main local application database	`crp`	`crp`	`5433` on the host
Integration test database	`crp_test`	test configuration	Test environment

The `crp_test` database is used for integration testing and is isolated from the main local application database. The production deployment uses only the main application database.

2.2 Production database parity

For production, a Render Managed PostgreSQL Free instance was provisioned with the database name explicitly set to `crp`. This maintains 1:1 parity with the local main database and avoids environment-specific database naming assumptions.

This parity ensures that:

- The same connection-string structure works in local and production environments.
- The existing PostgreSQL schema can be applied without database-name changes.
- The application's migration command behaves consistently across environments.
- Local-to-production troubleshooting is simpler because the database identity is the same.

The integration-test database `crp_test` remains a test-only database and is not used by the Render production service.

3. Repository and Containerization

3.1 Monorepo structure

This repository is an npm workspace monorepo. The root contains the workspace `package.json` and `package-lock.json`, while the web application is located in `apps/web/`. The web workspace depends on the root lockfile and workspace configuration during installation and build.

The relevant files are:

- `package.json`: root npm workspace configuration.
- `package-lock.json`: dependency lockfile.
- `apps/web/package.json`: web workspace scripts and dependencies.
- `apps/web/Dockerfile`: production image definition.
- `apps/web/server.mjs`: custom Next.js and Socket.IO server.
- `apps/web/db/schema.sql`: idempotent database schema.

3.2 Render Docker configuration

The Render Web Service was configured with the **Docker** runtime. Because the repository is a monorepo, the Docker build context had to remain at the repository root so that Docker could read the root workspace metadata and lockfile.

The Render settings are:

Setting	Value
Runtime	Docker
Root Directory	Blank
Dockerfile Path	`apps/web/Dockerfile`
Build context	Repository root
Service type	Render Web Service

The Root Directory was deliberately left blank. This makes the repository root the Docker build context. The Dockerfile path points to the application-specific Dockerfile inside `apps/web/`.

Render therefore performs the equivalent of:

```
docker build -f apps/web/Dockerfile .
```

The Dockerfile follows a multi-stage build:

1. Installs the root and web-workspace dependencies using the lockfile.
2. Builds the Next.js application.
3. Creates a smaller production runtime image.
4. Copies the required application, database, and migration files.
5. Runs the database migration before starting the application server.

The container command is equivalent to:

```
npm run db:migrate --workspace @ridepool/web
npm run start --workspace @ridepool/web
```

The migration step applies the idempotent schema from `apps/web/db/schema.sql` before the application starts. This allows a newly provisioned production database to be initialized automatically during deployment.

4. Render PostgreSQL Configuration

The Render Managed PostgreSQL Free instance was created with:

- Database name: `crp`
- A Render-managed PostgreSQL connection endpoint.
- An Internal Database URL available to services in the same Render environment.

The production web service does not connect through the public database endpoint. Instead, its `DATABASE_URL` is configured with Render's **Internal Database URL**. This keeps database traffic on Render's private network, avoiding unnecessary public-internet exposure and reducing network latency between the web service and database.

The password and full connection string are intentionally not included in this document. They are managed through Render's protected environment-variable configuration.

5. Environment Variable Injection

Environment variables were configured in the Render Web Service dashboard using the values from the local environment configuration as a starting point. Secrets were entered into Render's protected environment-variable interface rather than committed to the repository.

5.1 Required production variables

Variable	Production value or source	Purpose
`DATABASE_URL`	Render PostgreSQL **Internal Database URL**	Connects the web service to the managed `crp` database over Render's private network.
`APP_URL`	`https://campus-ridepooling-automation.onrender.com`	Defines the public application origin used in links, redirects, origin checks, and real-time connection handshakes.

5.2 Production service variables

The mail, Gemini, and payment-reminder variables below are used by the deployed service.

Variable	Purpose
`MAIL_SCRIPT_URL`	Google Apps Script endpoint for verification and password-reset email delivery. Required for new-account verification and password reset.
`MAIL_SCRIPT_SECRET`	Shared secret for the mail relay. Required alongside `MAIL_SCRIPT_URL`.
`GEMINI_API_KEY`	Server-side Gemini access for recommendations, chatbot responses, and advisory complaint analysis.
`GEMINI_MODEL`	Shared Gemini model identifier for recommendations, chatbot interpretation, and complaint analysis.
`PAYMENT_REMINDER_CRON_SECRET`	Bearer secret used to authorize the payment-reminder endpoint.

No secret values should be written to this document, committed to Git, or exposed in client-side variables.

All runtime Gemini features use the same model selector:

```
GEMINI_MODEL=gemini-3.1-flash-lite
```

The shared selector is implemented in `apps/web/src/lib/ai/gemini-config.ts`. Production deployments should define `GEMINI_MODEL` explicitly and provide `GEMINI_API_KEY` so valid AI requests are sent to Gemini. A fallback response is used only for explicit operational conditions such as a missing key, provider error, timeout, invalid response, or failed output validation.

5.3 Production URL

The deployed application is available at:

https://campus-ridepooling-automation.onrender.com

`APP_URL` was changed from the local value `http://localhost:3000` to this HTTPS production URL. This is required so that:

- Generated verification and password-reset links point to the deployed service.
- Origin and CSRF checks recognize the production browser origin.
- Socket.IO handshakes use the correct public origin.
- Redirects and server-generated application links do not point to localhost.

6. Deployment Procedure

The production deployment was completed with the following sequence.

Step 1: Confirm repository readiness

The application source, root npm workspace files, Dockerfile, database schema, and startup scripts were committed to the connected GitHub repository.

Step 2: Create the managed database

In Render:

1. Create a new **PostgreSQL** database.
2. Select the Free plan.
3. Set the database name to `crp`.
4. Wait for the database to become available.
5. Copy or reference the database's Internal Database URL for service configuration.

Step 3: Create the web service

In Render:

1. Create a new **Web Service**.
2. Connect it to the GitHub repository `CS455-Carpooling/Campus_Ridepooling_Automation`.
3. Select the branch used for deployment.
4. Select **Docker** as the runtime.
5. Leave **Root Directory** blank.
6. Set **Dockerfile Path** to `apps/web/Dockerfile`.
7. Configure the service environment variables.
8. Create the service and start the first deployment.

Leaving Root Directory blank is essential. If it were set to `apps/web`, Docker would lose access to the root `package.json` and `package-lock.json` required by the npm workspace installation.

Step 4: Configure environment variables

In the Render Web Service's environment settings:

1. Set `DATABASE_URL` to the Render PostgreSQL Internal Database URL.
2. Set `APP_URL` to `https://campus-ridepooling-automation.onrender.com`.
3. Add the optional mail, Gemini, and scheduler variables as required.
4. Save the environment configuration.

Step 5: Run the deployment

Render automatically builds the Docker image using the configured Dockerfile. The container then:

1. Installs production dependencies during the Docker image build.
2. Applies the PostgreSQL schema through `db:migrate` during container startup.
3. Starts the custom Next.js and Socket.IO server.

4. Exposes the service on the port supplied by the container environment.

Step 6: Verify the deployment

The deployment should be checked by:

- Opening the production URL in a browser.
- Confirming that public pages load over HTTPS.
- Checking the health endpoint:

  ```
  https://campus-ridepooling-automation.onrender.com/api/health
  ```

- Confirming that the response reports a healthy application.
- Checking Render logs for successful database migration and server startup.
- Testing registration, email verification, login, ride creation, ride joining, chat, and notifications as appropriate for the deployed configuration.
- A sample operations-admin account has been created for the demonstration environment. Credentials are intentionally not recorded in this document.

7. Requirement change and deployment limitations

The original D1 planning baseline named GCP as the intended cloud platform for `NFR-RD-18`. The implemented deployment uses Render.com Free tier instead. This decision was made on 5 November 2026 because Render supports the repository's Docker build directly from GitHub, provides managed PostgreSQL, and runs the custom server needed for Socket.IO without adding a separate real-time service. The platform change is recorded here rather than rewriting the submitted D1 requirement baseline.

Rejected alternatives included the originally planned GCP Cloud Run and Cloud SQL arrangement, which would require additional service configuration for this course deployment, and a serverless-only platform, which would not reliably maintain the application's WebSocket connections.

The Render Free tier has these operational limits relevant to the submission:

- The free web service sleeps after 15 minutes without traffic and may take about one minute to wake. Open the production site shortly before a demonstration.
- The workspace includes 750 free instance hours per month.
- The free PostgreSQL instance provides 1 GB of storage.
- The free PostgreSQL database created on 5 November 2026 expires after 30 days (5 December 2026) and is deleted 14 days later (19 December 2026) under the provider's stated free-tier policy. It has no backups.
- After the course demonstration, preserve the schema and deployment configuration and migrate to a paid or separately backed-up database if the service must remain available.

The deployment uses the mail, Gemini, and payment-reminder variables in Render. `MAIL_SCRIPT_SECRET` is managed in Render; its rotation history is not recorded in this repository.

8. CI/CD and Release Automation

The Render Web Service is linked directly to the GitHub repository. Automatic deployments are enabled for pushes to the configured `master` branch.

The release flow is:

```
Git push
    |
    v
Render detects the new commit
    |
    v
Docker image is rebuilt from the repository root
    |
    v
Database migration runs during container startup
    |
    v
New application container is deployed
```

This provides repeatable deployments without manually building or uploading an image. Render replaces the running service using its deployment process, providing zero-downtime deployment behavior subject to the platform's health checks and available Free-tier resources.

9. WebSocket and Real-Time Infrastructure

The application uses Socket.IO for real-time ride coordination, chat, and notifications. It requires a continuously running server process that can maintain long-lived WebSocket connections.

Render was selected because its Docker-based Web Service runs the application server directly and supports Socket.IO without a separate real-time service. The Free tier may sleep after inactivity, so the first request after an idle period can take longer while the service wakes. Standard serverless platforms commonly terminate or do not preserve WebSocket connections between invocations, which would require an additional real-time service or a more complex infrastructure design.

The Render deployment does not require a separate NGINX reverse proxy or custom load-balancer configuration for the current single-service deployment. The custom server in `apps/web/server.mjs` serves Next.js and Socket.IO from the same continuously running process.

10. Security and Operational Notes

- Production database traffic uses Render's Internal Database URL.
- Database credentials and API keys are stored as protected Render environment variables.
- Secrets are not included in this documentation.
- `APP_URL` uses HTTPS in production.
- The production database is separate from local and integration-test databases.
- The idempotent migration process avoids requiring manual schema setup on every deployment.
- The Free tier has resource, sleep, and availability limitations; these should be considered when interpreting first-request latency and real-time behavior.
- The database currently permits the provider's default external networking configuration; production hardening should restrict the allow list if external administration is no longer needed.

- Backups, retention, scaling, and uptime guarantees depend on the selected Render plan and should be reviewed before production use beyond the course submission.

11. Troubleshooting

Docker build cannot find workspace files

Verify that:

- Root Directory is blank.
- Dockerfile Path is exactly `apps/web/Dockerfile`.
- The repository root contains `package.json` and `package-lock.json`.

Database connection fails

Verify that:

- `DATABASE_URL` uses Render's Internal Database URL.
- The database name is `crp`.
- The PostgreSQL instance is available.
- The web service and database are in the expected Render environment.

Links or Socket.IO connections use localhost

Verify that `APP_URL` is set to:

```
https://campus-ridepooling-automation.onrender.com
```

Then redeploy or restart the web service so the updated environment is loaded.

Migration fails during startup

Review the Render deployment logs and confirm that:

- PostgreSQL is accepting connections.
- `DATABASE_URL` is valid.
- The service can reach the database over Render's private network.
- The database user has permission to create and modify the application schema.

12. Deployment Summary

The final deployment uses a Render Docker Web Service and a Render Managed PostgreSQL Free database named `crp`. The blank Root Directory and explicit `apps/web/Dockerfile` path preserve the npm workspace monorepo build context. Render's Internal Database URL provides private service-to-database communication, while the production `APP_URL` ensures correct HTTPS routing and WebSocket origin behavior. GitHub-linked automatic deployments provide repeatable CI/CD whenever the configured main/master branch changes.